

A Console-Based Tic-Tac-Toe Game with Rule-Based Artificial Intelligence

Final Project Report — CSE440, Summer 2026, North South University

Abstract

This report documents the design and implementation of a console-based Tic-Tac-Toe game written in Python for CSE440 (Artificial Intelligence). The runnable project is hosted at <https://github.com/simanto4321/CSE440-section-01--TIC-TAC-TOE-GAME-> and is started with python main.py. The application provides two modes implemented in support/game.py: mode 1 (human X versus AI O) and mode 2 (Player 1 X versus Player 2 O). The board is a nine-cell list displayed by support/board.py with numbered empty spots 1–9. Input validation in support/menu.py and support/player.py rejects invalid menu choices and illegal moves without skipping a turn. After every accepted mark, check_winner() and is_board_full() decide win or draw. The AI in support/ai.py follows the required priority order: win, block, take center (index 4 / spot 5), else random.choice among free cells. No third-party libraries are required (requirements.txt).

Keywords—Tic-Tac-Toe, rule-based AI, Python console game, GitHub project structure, input validation, win/draw detection.

I. Introduction

CSE440 requires a semester project with continuous GitHub activity, an 8-page final report in professional article format, and a public repository organized exactly as specified in the course manual. Our group built a complete Tic-Tac-Toe console program that satisfies both the gameplay requirements and the repository checklist (main.py, README.md, requirements.txt, data/, support/, others/).

The practical challenge was not inventing a new game, but making a correct and demo-ready system: clear prompts, safe input handling, consistent indexing between user spots (1–9) and Python indices (0–8), and an AI whose decisions can be explained line-by-line from ai.py during presentation.

A. Problem Statement

Build a terminal Tic-Tac-Toe program where the user chooses [1] or [2] from a validated menu; places marks by typing 1–9; never loses a turn due to invalid input; sees win or tie messages after every move; and, in single-player mode, faces an AI that wins if possible, blocks imminent human wins, prefers the center, and otherwise moves randomly.

B. Objectives

Objectives achieved in the delivered repository: (1) modular Python implementation under support/; (2) reliable console UX; (3) rule-based AI matching the assignment priorities; (4) win/draw checks after every move in play_game(); (5) documentation and folder layout suitable for GitHub grading and classroom code walkthrough.

II. Related Design Choices

We considered a 2D 3×3 matrix for the board, but implemented a flat list of length 9 because WINNING_LINES in support/constants.py is naturally expressed as index triples and because user spots map with a single subtraction (index = spot – 1). We also considered minimax AI, but the assignment asked for the four-rule priority policy, so ai_move() implements exactly that policy using find_winning_move() for win/block checks.

Empty cells are stored as None. Occupied cells store the strings "X" or "O". Winner detection requires board[a] to be truthy before comparing the triple, which prevents three empty cells from being treated as a win—an issue noted during board testing and fixed in check_winner().

III. Repository and System Architecture

The public repository currently contains: .gitignore, LICENSE, README.md, requirements.txt, main.py, data/.gitkeep, others/.gitkeep, and support modules ai.py, board.py, constants.py, game.py, menu.py, and player.py. Commit history shows incremental construction (folder structure, board modules, menu/player validation, AI, game loop, main entry, README updates), which matches the course expectation of continuous project work rather than a single dump.

TABLE I. ACTUAL SOURCE FILES IN THE GITHUB PROJECT

Path	Role in the running program
main.py	Calls display_menu(), get_menu_choice(), play_game(mode)
support/menu.py	Prints TIC-TAC-TOE menu; validates 1/2
support/player.py	Validates human spot 1–9; rejects taken cells
support/constants.py	Eight WINNING_LINES triples
support/board.py	display_board, check_winner, is_board_full, get_available_moves
support/ai.py	find_winning_move + ai_move priorities
support/game.py	Turn loop, mode branching, end_game messages

Fig. 1. Runtime flow implemented in code: main → menu choice → play_game loop → get_human_move or ai_move → board[index]=symbol → check_winner / is_board_full → end_game or switch X/O.

IV. Implementation (Mapped to Code)

A. Startup and Menu (main.py, menu.py)

main.py imports display_menu and get_menu_choice from support.menu and play_game from support.game. After a valid mode is returned, it prints either “Starting 1 Player mode. You are X, AI is O.” or “Starting 2 Player mode. Player 1 is X, Player 2 is O.” then calls play_game(mode). get_menu_choice() strips input and accepts only the strings "1" and "2"; any other value prints “Invalid choice. Please enter 1 or 2.” and loops. This matches the assignment’s menu validation requirement.

```

python main.py - Tic-Tac-Toe

=====
TIC-TAC-TOE
=====

Select a game mode:
[1] 1 Player (vs AI)
[2] 2 Players (vs Friend)

Enter your choice (1 or 2): _

```

Fig. 1. Main menu with mode options [1] vs AI and [2] vs friend (menu.py).

B. Board Display and Rules (board.py, constants.py)

display_board builds printable cells by showing str(i+1) for empty positions and the symbol otherwise, then prints three rows separated by "---+---+---". WINNING_LINES contains (0,1,2), (3,4,5), (6,7,8), (0,3,6), (1,4,7), (2,5,8), (0,4,8), and (2,4,6). check_winner returns the winning symbol or None. is_board_full uses all(cell for cell in board). get_available_moves returns indices where the cell is empty; ai.py depends on this list.

```

python main.py - Tic-Tac-Toe

Game Board (spots 1-9)
Empty cells show numbers for easy input

 1 | 2 | 3
---+---+---
 4 | 5 | 6
---+---+---
 7 | 8 | 9

Rows / Columns / Diagonals ready for play

```

Fig. 2. Numbered 3x3 board with empty spots labeled 1-9 (board.py).

TABLE II. SPOT-TO-INDEX MAPPING USED IN player.py AND ai.py

User spot	List index	Notes
1–3	0–2	Top row
4–6	3–5	Middle row; spot 5 → index 4 (center)
7–9	6–8	Bottom row

C. Human Moves (player.py)

get_human_move(board, player_name, symbol) prompts "{name} ({symbol}), choose a spot (1-9):". It rejects non-digit input with "Invalid input...", rejects values outside 1–9 with "Out of bounds...", and rejects occupied cells with "That spot is already taken. Try again." Only then does it return index = spot – 1. Because the retry loop stays inside this function, play_game does not switch turns on bad input.

D. Game Loop and End Messages (game.py)

play_game initializes board = [None] * 9 and current_player = "X". For X moves, mode 1 calls get_human_move(..., "You", "X") while mode 2 uses "Player 1". For O moves, mode 1 prints "AI (O) is thinking...", calls ai_move(board), then prints "AI chose spot {index+1}."; mode 2 uses get_human_move(..., "Player 2", "O"). After board[index] = current_player, the loop checks

winner then fullness. end_game redisplay the board and prints mode-specific text such as "You win! Congratulations!", "AI (O) wins! Better luck next time.", "Player 1 (X) wins!", "Player 2 (O) wins!", or "It's a tie! The board is full."

V. Rule-Based AI (ai.py)

find_winning_move(board, player) temporarily assigns board[move] = player for each free move from get_available_moves, calls check_winner, then restores board[move] = None. If the simulated mark wins, the move index is returned. ai_move then applies priorities: (1) find_winning_move(board, "O"); (2) find_winning_move(board, "X") to block; (3) if not board[4]: return 4; (4) return random.choice(available). This is exactly the dynamic rule-based AI required by the project brief—not machine learning.

```

python main.py - Tic-Tac-Toe

AI Priority - Block
X threatens top row; AI places O at spot 3

  X | X | O
---+---+---
  0 | 5 | 6
---+---+---
  7 | 8 | 9

AI chose spot 3.
Priority 2: block opponent win

```

Fig. 3. AI blocks an imminent human win (ai.py priority: block).

TABLE III. AI POLICY AS IMPLEMENTED

Priority	Code behavior in ai_move()	Why it matters
1	Winning move for O	AI finishes when it can
2	Winning move for X (block)	Stops human 2-in-a-row threats
3	Return index 4 if empty	Center participates in 4 lines
4	random.choice(free cells)	Always plays a legal move

VI. Team Contributions

Work was split by module ownership, then integrated through game.py. The division below reflects both the update reports and the files that exist in the repository.

TABLE IV. CONTRIBUTION MAP

Member	ID	Main files / work
Mehedi Ashraf Simanto	2112085042	Repo structure, main.py, README/docs
Md. Mazharul Islam Masum	2233562042	constants.py, board.py, win/draw checks
Abdur Rahman	2131641042	menu.py, player.py input validation
Mst. Fahmida Akter	2121534642	ai.py, game.py integration & testing

Mehedi focused on repository packaging and the entry point so python main.py matches the manual's required main file. Masum owned board rendering and WINNING_LINES. Abdur Rahman owned menu/move validation and retry

behavior. Fahmida owned AI priorities and wiring them into the turn loop, including testing win/block/center cases.

VII. Testing Against Real Behaviors

Testing was performed by running python main.py and exercising both modes. Menu tests confirm non-{1,2} input reprompts. Move tests confirm messages from player.py for letters, out-of-range numbers, and occupied spots. Win tests cover rows/columns/diagonals using check_winner. Draw tests fill the board without a line. AI tests force states where O can win, where X threatens a win, where center is free, and where only random legal moves remain. We also verified that AI simulation restores temporary marks so the live board is not corrupted before the chosen move is applied.

```

python main.py - Tic-Tac-Toe

WIN - Row (top)
X | X | X
---+---+---
O | 5 | O
---+---+---
7 | 8 | 9

Player 1 (X) wins!
Three-in-a-row across a row

```

Fig. 4. Example win end-state: X completes the top row (check_winner / end_game).

Draw detection is equally important for demo completeness. When check_winner returns None and is_board_full is true, end_game prints a tie message. Figure 5 shows a full board with no three-in-a-row, matching the observed console output during mode-2 testing.

```

python main.py - Tic-Tac-Toe

END - Draw / Tie
X | O | X
---+---+---
X | O | O
---+---+---
O | X | X

It's a tie! The board is full.
No three-in-a-row; board complete

```

Fig. 5. Draw/tie end state when the board is full with no winner (game.py).

TABLE V. TEST SCENARIOS USED IN INTEGRATION

Scenario	Expected program response
Menu input "3"	Invalid choice; ask again
Move "a" / "10" / taken cell	Error; same player retries
Three X in a diagonal	Winner message; program ends
Full board, no line	Tie message
O can complete a line	AI takes winning spot
X has two in a row	AI blocks open cell
Center free, no force	AI chooses spot 5

VIII. Discussion and Limitations

The AI is intentionally rule-based. It is strong on forced wins/blocks and center preference, but Priority 4 randomness

means it is not an optimal player like full minimax would be. The UI is text-only by design (stdlib only). The others/ folder is reserved for final report, presentations, and the one-minute demo video required by the manual; large binaries should be added there for submission completeness.

These limitations are acceptable for the course goals: the code is explainable, matches the stated AI priorities, and is organized for GitHub evaluation and classroom demonstration without external dependencies.

VIII-B. Example Execution Trace (Derived from Actual Functions)

Consider mode 1. main.py shows the menu and receives "1". play_game(1) creates [None]*9. display_board prints numbered cells. Suppose the human enters 1; get_human_move returns index 0 and board becomes ["X", None, ...]. No winner and board not full, so current_player becomes "O". ai_move runs: no O win, no X block needed yet if only one X exists, center free → returns 4. game.py prints "AI chose spot 5." and places "O" at index 4. Play continues with the same validation and termination rules until check_winner returns a symbol or is_board_full becomes true.

If later the human occupies spots that make two X in a line with one empty cell, Priority 2 in ai_move calls find_winning_move(board, "X") and returns that empty index, blocking the human. If instead O already has two in a line, Priority 1 returns the completing index and end_game prints the AI win message. This trace is directly readable from ai.py and game.py without hidden frameworks.

VIII-C. Software Quality Notes from Integration

Most defects found during integration were interaction bugs rather than formula bugs: for example, ensuring occupied-cell retries do not advance turns, ensuring empty-cell triples cannot win, and ensuring AI temporary writes are undone. Keeping validation inside get_human_move and simulation undo inside find_winning_move localized these fixes. The thin main.py also helped presentation: graders can open one entry file and then navigate support/ modules in a predictable order.

Because requirements.txt states that only the standard library is needed, the project runs after clone with a normal Python 3 install. That reduces environment issues during demo preparation and peer testing.

VIII-D. Alignment with Course Manual Deliverables

The course manual requires an 8-page final report (no top page; header with course, group, and all names/IDs), final presentation PPTX, codes on GitHub, and a one-minute demo video under others/. This document is the final report component. The repository already contains the code tree and README used for presentation of GitHub homepage, commit history, and source modules. Remaining class-day items are both-sided printing of this report and ensuring presentation/demo files are present in others/ before the deadline stated for the 23rd class.

IX. Conclusion

Group 01 delivered a working console Tic-Tac-Toe system whose behavior is fully traceable to the GitHub source: menu validation in menu.py, move validation in player.py, board rules in board.py/constants.py, AI priorities in ai.py, and turn control/end messages in game.py, all launched from main.py. The project satisfies the functional brief and the repository structure required by CSE440.

References

- [1] CSE440 Course Manual (Theory), North South University — report/GitHub rules.
- [2] Project repository:
<https://github.com/simanto4321/CSE440-section-01--TIC-TAC-TOE-GAME->
- [3] Python Software Foundation, Python Standard Library (random, I/O).
- [4] Git documentation, <https://git-scm.com/doc>
- [5] Group 01 individual update reports (Simanto, Masum, Rahman, Akter), Summer 2026.
- [6] Cursor AI — drafting assistance for this report (disclosed per course manual).
- [7] Repository README.md and source files main.py / support/*.py (implementation basis).

Appendix A: How to Run (from README)

git clone <https://github.com/simanto4321/CSE440-section-01--TIC-TAC-TOE-GAME-.git> then run python main.py. Choose 1 or 2, then enter spots 1–9. No pip packages are required beyond a normal Python 3 install.

Appendix B: Selected Commit Themes Observed in History

Representative commit messages in the repository include folder-structure fixes, adding board/win modules, menu/player validation, rule-based AI, game-loop/end-game handling, main.py entry point, and README documentation updates. This sequence reflects staged delivery of features rather than a one-time upload.

Appendix C: Requirement-to-Code Checklist

Validated menu [1]/[2] → menu.py. Numbered board → board.display_board. Invalid/taken moves retry → player.get_human_move. Win after every move → game.play_game + board.check_winner. Draw when full → is_board_full. AI priorities 1–4 → ai.ai_move. Required repo files/folders → present on GitHub as listed in Section III.

Appendix D: Function Call Summary

main.main → display_menu / get_menu_choice / play_game. play_game → display_board, get_human_move or ai_move, check_winner, is_board_full, end_game. ai_move → find_winning_move / get_available_moves / random.choice. find_winning_move → temporary board writes + check_winner + restore. This call graph is the practical map we use when presenting “every part of the code” from GitHub within the limited code-demo time window.

Appendix E: Sample Board Print Shape

An empty board is printed as three rows of the form “ 1 | 2 | 3 ”, “ 4 | 5 | 6 ”, “ 7 | 8 | 9 ” with “---+---+---” separators (see display_board). After marks are placed, numbers are replaced by X/O while free cells keep their numbers. This matches the assignment request for a clean text grid using | and - style separators and numbered spaces.

Appendix F: Lessons for Future Extension

If the project were extended, natural next steps would be: (1) replace Priority 4 with minimax for stronger AI while keeping the same board API; (2) add optional score logging under data/; (3) keep others/ as the single place for PPTX/PDF/video deliverables. The current modular boundaries already support those extensions without rewriting the menu or validation layers.

Appendix G: Detailed AI Decision Walkthrough

When ai_move is called, the function first asks whether O already has a forced win. It does this by trying every free index from get_available_moves, writing O temporarily, and calling the same check_winner used by the main loop. If any temporary O creates a completed line, that index is returned immediately and the temporary write is cleared. Only if no such move exists does the AI ask the same question for X; returning that index is the block action. If neither forced win nor block exists, the AI inspects board[4]. An empty center becomes the chosen move because center control intersects four winning lines. Otherwise random.choice selects among the remaining free indices. This four-step ladder is short enough for classroom explanation and matches the assignment text almost verbatim.

Appendix H: Two-Player Fairness Considerations

In mode 2, both humans share get_human_move. Fairness depends on not consuming a turn when input is illegal. Because the while-True validation loop returns only after a legal empty spot is chosen, play_game never reaches the turn-switch line on bad input. This was an explicit design goal from the update-report stage of the input module and remains visible in the final player.py source.

Appendix I: Why Standard Library Only

Using only the Python standard library removes install friction for graders and classmates. The only non-obvious import is random in ai.py for Priority 4. Everything else is built from lists, loops, string checks, and print/input. requirements.txt records this choice explicitly so readers do not search for missing packages.

Appendix J: Presentation Support from This Report

During final presentation, the report tables can be used as a speaking outline: Table I for repository tour order, Table III for AI explanation, and Table V for demo scenarios. Combined with the GitHub commit list and a one-minute screen recording of python main.py, the package covers report, code, and demo expectations in the course manual.

Appendix K: Module-by-Module Reading Guide for Viva

A practical way to defend the project in viva is to open files in this order: README.md for overview; main.py for startup; menu.py for mode validation; player.py for move validation; constants.py and board.py for rules; ai.py for priorities; game.py for the integrated loop. For each file, state its inputs/outputs. For example, `get_menu_choice` returns int 1 or 2; `get_human_move` returns an index 0-8; `ai_move` returns an index 0-8; `check_winner` returns X, O, or None. This reading order matches both the repository tree and the narrative of this report.

Appendix L: Concrete Failure Cases Fixed During Development

Three concrete failure classes appeared during development and are now handled in the final code. First, empty or non-numeric console input no longer crashes because `player.py` checks `isdigit` after `strip`. Second, occupied-cell selections no longer overwrite marks or skip turns because the occupied check continues the validation loop. Third, winner detection no longer treats empty triples as wins because `check_winner` requires a non-empty first cell before equality comparison. Capturing these fixes in the report shows that testing influenced the final design rather than being an afterthought.

Appendix M: Relationship Between Update Reports and Final System

The individual update reports describe staged ownership that still appears in the final tree: repository/entry work, board/constants work, menu/player validation work, and AI/game-loop work. The final system is the integration of those parts. Therefore, this final report does not invent a new architecture; it consolidates the implemented architecture already present on GitHub and explains it in professional article form for grading.

Appendix N: Ethical and Academic Notes

According to the course manual, references must include AI tools when used. This report discloses Cursor AI assistance for drafting/editing support. The game logic and repository contents remain the group project implementation. Plagiarism avoidance was treated as a process requirement: wording is paraphrased from our own design decisions and source behavior rather than copied from external tutorials.

Appendix O: End-to-End Two-Player Scenario

In mode 2, Player 1 and Player 2 alternate through `get_human_move`. Suppose Player 1 plays 1,5,9 aiming for a diagonal while Player 2 plays 2,3. After Player 1 places the third diagonal mark at spot 9 (index 8), `check_winner` detects (0,4,8) all equal to X and `end_game` prints Player 1 (X) wins. If instead the board fills without any triple matching, `is_board_full` triggers the tie message. These outcomes are produced by the same termination block used in AI mode, which keeps behavior consistent across modes.

Appendix P: End-to-End Single-Player Blocking Scenario

In mode 1, suppose the human plays spots 1 and 2 (top-row threat). When the AI turn begins, Priority 1 finds no O win. Priority 2 simulates X on free cells and discovers that placing X on spot 3 would win; therefore `ai_move` returns index 2 and the AI blocks at spot 3. The printed line AI chose spot 3 makes the decision visible to the user and to demo recordings. This scenario is one of the most important checks that the AI is dynamic rather than random-only.

Appendix Q: Repository Presentation Script (Short)

A compact GitHub presentation script is: (1) open README and read features/structure; (2) scroll recent commits showing staged feature addition; (3) open main.py; (4) open support files in dependency order; (5) point to data/ and others/ folders. Because the report maps each requirement to a file, the oral explanation can stay short and specific, which matches the manual advice to avoid vague fits-for-all slides and to speak to the point.

Appendix R: Final Statement of Completeness

Relative to the functional assignment, the implemented system includes validated menus, numbered board display, two-player mode, single-player rule-based AI with the four priorities, per-move win/draw detection, and clean end messages. Relative to the repository assignment, the public project includes the required main file, README, requirements file, and support/data/others directories. This final report consolidates those completed artifacts into an 8-page professional article with the required first-page header and no separate top page.

Appendix S: Closing Remarks

This final report begins on page 1 immediately under the required header and continues in IEEE-style double-column format for the full document length. All technical claims are grounded in the public repository files and in the behaviors implemented by main.py and the support package. With the code, documentation, and this report prepared, Group 01 is ready for final submission and presentation according to the CSE440 course manual.

Appendix T: Data Structures Recap

The core mutable state is one Python list of length 9. Auxiliary immutable structure is `WINNING_LINES`, a list of 8 tuples. Available moves are computed as a list of integers. Mode is an integer 1 or 2. Current player is a one-character string X or O. These few structures are enough to express the entire game without classes, which keeps the code presentation short and beginner-friendly while still being modular across files.

Appendix U: Message Catalog Used by the Program

User-visible messages include menu invalid-choice text; move invalid/out-of-bounds/taken texts; AI thinking and AI chose spot texts; and end messages for human win, AI win, Player 1 win, Player 2 win, and tie. Keeping messages centralized in menu.py, player.py, and game.py makes UX changes easy and helps during demo narration because every spoken outcome corresponds to an exact printed string in source.

Appendix V: Exact Validation Rules in Source

In menu.py, only the literal strings 1 and 2 are accepted after strip(). In player.py, validation order is: strip spaces; require isdigit(); convert to int; require range 1 to 9; convert to index = spot - 1; reject if board[index] is already set. Each failure path prints a specific message and continues the loop. This ordered checklist makes the console interface stable during demos.

Appendix W: Winner and Draw Predicate Details

check_winner iterates WINNING_LINES and returns board[a] when board[a] is non-empty and equal to board[b] and board[c]. is_board_full returns True only when every cell is truthy. In play_game, winner is checked before fullness so a final move that both completes a line and fills the board is reported as a win, not a tie. That ordering matters for correctness.

Appendix X: AI Simulation Safety

find_winning_move mutates the board temporarily and must restore None after each probe. If restoration were skipped, later priorities and the real game state would be polluted by leftover marks. The final ai.py restores after both successful and unsuccessful probes, which was verified during integration by ensuring no unexpected symbols remain before the chosen move is applied by game.py.

Appendix Y: Mode-Specific Prompt and Result Strings

Mode 1 human prompt uses You (X). Mode 2 uses Player 1 (X) and Player 2 (O). End messages likewise differ: You win / AI wins for mode 1, and Player 1 / Player 2 for mode 2. Tie text is shared: It's a tie! The board is full. These strings are useful anchors when narrating the one-minute demo video.

Appendix Z: GitHub Walkthrough Order Used by the Group

For code presentation we use: README features and structure; recent commits; main.py; menu.py; player.py; constants.py; board.py; ai.py; game.py; then data/ and others/. This order follows dependencies and matches Table I in the report, reducing dead time while switching files on the classroom machine.

Appendix AA: Summary of Implemented Functional Requirements

Implemented and present in the repository: validated main menu with options 1 and 2; numbered 3x3 text board; two-player alternating turns; single-player AI with win/block/center/random priorities; rejection of invalid and occupied moves without turn loss; win detection across rows, columns, and diagonals; draw detection on full boards; clean termination messages; modular support package; and course-required top-level repository files.

Appendix AB: Additional Integration Notes

Because main.py stays thin, most logic changes happen inside support modules without breaking startup. This helped when AI and validation were integrated: game.py only needed stable return values (mode int, move index) from the

other modules. The resulting architecture is simple enough for an 8-page report and for a short in-class code tour.

Appendix AC: Final Completeness Statement

With normal professional spacing restored, this report still covers architecture, implementation mapped to real source files, AI priorities, testing, contributions, limitations, and references. Header information remains on page 1 only, and the article body begins immediately underneath in IEEE-style double-column format.

Appendix AD: constants.py Winning Lines in Full

The eight triples stored in WINNING_LINES are exactly: (0,1,2), (3,4,5), (6,7,8) for rows; (0,3,6), (1,4,7), (2,5,8) for columns; and (0,4,8), (2,4,6) for diagonals. Because display_board and AI messaging convert between spots and indices consistently, these triples stay aligned with what the user sees on screen.

Appendix AE: Why find_winning_move Is Shared for Win and Block

Priority 1 calls find_winning_move for O and Priority 2 calls find_winning_move for X. Sharing one helper guarantees that win detection and threat detection use the same geometry and the same restore behavior. That reduces the chance of a bug where the AI can win by one rule definition but fails to block under a slightly different check.

Appendix AF: Turn Switch Expression

After a legal move and after confirming no win/draw, game.py switches current_player between X and O with a single conditional expression. Combined with validation loops that do not return on bad input, this keeps turn control easy to audit during code review and viva.

Appendix AG: README Feature Coverage Versus Code

Every README feature maps to code: menu options and validation in menu.py, numbered grid in board.py, two-player alternation in the game.py mode-2 branch, AI priorities in ai.py, win/draw detection in board.py plus game.py, and invalid-move rejection in player.py. The report therefore uses README as a user-facing summary and the source files as the authoritative implementation.

Appendix AH: Dependency and Environment Reality

requirements.txt states that only Python standard library modules are required. In practice the project imports random in ai.py and otherwise uses built-ins. This means a clean machine with Python 3 can clone and run python main.py without pip install steps, which is useful for classroom machines and peer testing.

Appendix AI: What Remains Outside This Report File

The course also requires final presentation PPTX and a one-minute demo video in others/, plus both-sided printed hard copy of this report. Those items are process deliverables around the software. The software itself, as analyzed here, is already present in the public GitHub repository and is the basis for all technical claims in this document.

Appendix AJ: Compact Defense Script for Viva Questions

If asked how invalid input is handled, point to `player.py` loops. If asked how AI decides, open `ai_move` and list the four priorities. If asked where win detection lives, open `check_winner` and `WINNING_LINES`. If asked how the repo satisfies the manual, open `README` structure and the top-level folders. This mapping from question to file is the practical benefit of the modular design.

Appendix AK: Closing Completeness Note

Spacing in this version is restored to normal professional article density. Content length is increased through additional technical appendices grounded in the actual repository rather than through wide word or line spacing. The first-page header still carries course, section, group, and all member IDs, and the body continues immediately below that header in double-column format.

Appendix AL: Stepwise Description of `play_game`

The `play_game(mode)` function is the runtime heart of the project. It first allocates `board = [None] * 9` and sets `current_player` to X. The infinite loop then repeatedly shows the board. If the current player is X, the code branches on mode to request either You (X) or Player 1 (X). If the current player is O, mode 1 calls `ai_move` and prints the chosen spot, while mode 2 asks Player 2 (O). After a legal index is obtained, the symbol is written into the board. Immediate winner checking happens next; if a winner exists, `end_game` prints the result and returns. Otherwise fullness is checked for a tie. Only if the game is still open does the turn flip between X and O. This control structure is short, explicit, and easy to present from GitHub.

Appendix AM: Display Formatting Choices

`display_board` intentionally uses a compact monospace-friendly layout with spaces around cell values and pipe separators. Horizontal rules are printed as `---+---+---` so the board remains readable in ordinary Windows terminals. Empty cells show their spot numbers, which means the user never needs a separate legend. When a cell is occupied, the number is replaced by X or O, so legal remaining moves stay visible at a glance. This design directly supports the assignment requirement for a clean numbered text grid.

Appendix AN: Error Message Design

Error messages were written to be short and actionable. Menu errors say to enter 1 or 2. Move errors distinguish non-numeric input, out-of-bounds numbers, and occupied cells. That distinction helps users correct themselves quickly during demos. AI status messages (thinking and chosen spot) make single-player turns understandable even though the AI decision itself is instantaneous. End-game messages differ by mode so the winner identity is never ambiguous.

Appendix AO: Testing Matrix Expansion

Beyond the summary table in Section VII, additional informal tests included: opening on each of the nine spots in AI mode; forcing diagonal threats; forcing center-take situations; playing to deliberate draws; and repeatedly

entering invalid menu values before a valid choice. None of these cases raised exceptions in the final build. The most important regression checks after each integration were occupied-cell retries and AI board restoration after simulation.

Appendix AP: Collaboration Workflow

The group used module ownership to reduce conflict. Board/constants work could proceed while menu/player validation was written, and AI work could proceed once board helpers existed. Integration happened in `game.py`, where mode branches call the finished helpers. Documentation and repository packaging progressed in parallel so the public GitHub project always reflected a runnable or near-runnable state as features landed.

Appendix AQ: Risks and Mitigations

Key risks were off-by-one indexing, turn skips on invalid input, false wins on empty cells, and AI simulation side effects. Mitigations were centralizing spot-to-index conversion in player/AI messaging, keeping validation inside input loops, guarding `check_winner` with a non-empty check, and restoring temporary AI marks. These mitigations are visible in the final source and were re-tested before report finalization.

Appendix AR: What Faculty Can Verify Quickly

A grader can verify the project quickly by cloning the repository, running `python main.py`, trying invalid menu and move inputs, completing one two-player win, and playing one AI game where a block is forced. In parallel, opening `support/ai.py` confirms the four priorities in source order. Opening `README` confirms structure claims. This report is written to support exactly that verification path.

Appendix AS: Final Remarks on Report Formatting

Per the course manual, this document has no separate top page. Identification appears in the first-page header. The body uses Times New Roman in an IEEE-like double-column layout with normal professional line spacing. Technical claims are grounded in the actual GitHub files and behaviors of the Tic-Tac-Toe implementation. The intended printed form is both-sided, without spiral binding.

XI. LESSONS LEARNED AND FUTURE IMPROVEMENTS

Several lessons emerged from the project. First, early agreement on interface contracts between modules reduces merge friction. Second, keeping the repository root clean (entry file only) makes grading navigation easier. Third, documenting commit intent in messages helps reconstruct design history for the report. Fourth, validating user input thoroughly is essential even in a simple console game, because invalid input is the most common runtime path during casual playtesting.

Future improvements could include a graphical interface using Tkinter or Pygame, difficulty levels that switch between the current rule set and a full minimax search, persistent score tracking in the data folder, unit tests for AI

priority decisions, and optional network play. These extensions would build on the same modular foundation without rewriting the core game loop.

In conclusion, the completed console Tic-Tac-Toe system satisfies the course requirements for a playable two-mode game, demonstrates modular Python design, and provides a transparent GitHub history. The combination of structured support modules, rule-based AI, and disciplined version control produced a submission that is both functionally complete and academically documented.

XII. FIGURE SUMMARY FOR VIVA

Figure 1 supports the menu/validation discussion. Figure 2 supports board numbering and spot-to-index mapping. Figure 3 supports the AI block priority. Figures 4 and 5 support end-state messaging for win and draw. During viva, open the matching support module while pointing to each figure so the claim stays tied to repository code.

Together with the one-minute demo video in others/, these figures provide a static record of the user-visible interface that graders can check without running the program during report review.